HANDS-ON AWS PROJECT

Restricting DynamoDB Access with a Customer-Managed KMS Key

linkedin.com/in/mohammed-abdul-rasheed

Full DynamoDB permissions shouldn't automatically mean full data access. This project attaches a customer-managed KMS key to a DynamoDB table's encryption at rest, and proves — with a live scan from a second IAM user — that without explicit key permissions, the data stays unreadable even to someone with complete DynamoDB access.

1 Customer Managed Key | **1 Key Policy** | **1 DynamoDB Table** | **2 IAM Users** | **Verified via Console**

HIGH-LEVEL FLOW

AWS · KMS · DYNAMODB · IAM · ENCRYPTION AT REST

Mohammed Abdul Rasheed | linkedin.com/in/mohammed-abdul-rasheed

Project Overview

By default, DynamoDB encrypts data at rest with an AWS-owned key, invisible to IAM policy — anyone with table permissions can read the data. This project swaps that for a **customer-managed KMS key**, whose key policy names exactly one IAM user as authorized to encrypt and decrypt with it. The table is then re-encrypted under that key, and a second IAM user — who has **full DynamoDB permissions** but no mention in the key policy — is used to prove that table access alone no longer guarantees data access.

Component	Name	Configuration
KMS Key	rasheed-key	`93f05216-fa06-4706-a16b-34efda74eff5`
Key Type	Symmetric	SYMMETRIC_DEFAULT · Encrypt and decrypt
Key Policy	Scoped statement	Grants Encrypt/Decrypt/ReEncrypt/GenerateDataKey/DescribeKey to one user
Authorized IAM User	RASHEED	Named principal in the key policy
Unauthorized IAM User	rasheed2	Full DynamoDB access, no KMS grant
DynamoDB Table	kms-test	Encryption at rest → Customer managed key

> **Why this matters:** IAM policies and KMS key policies are two separate permission boundaries. Attaching a customer-managed key to a resource means both boundaries have to agree — a user can have every DynamoDB action allowed and still be completely blocked from reading the data if the key itself never named them. This turns encryption into an actual access-control layer, not just a compliance checkbox.

Key policy = real gatekeeper · **Table permissions ≠ data access** · **Enforced by AWS KMS, not app logic**

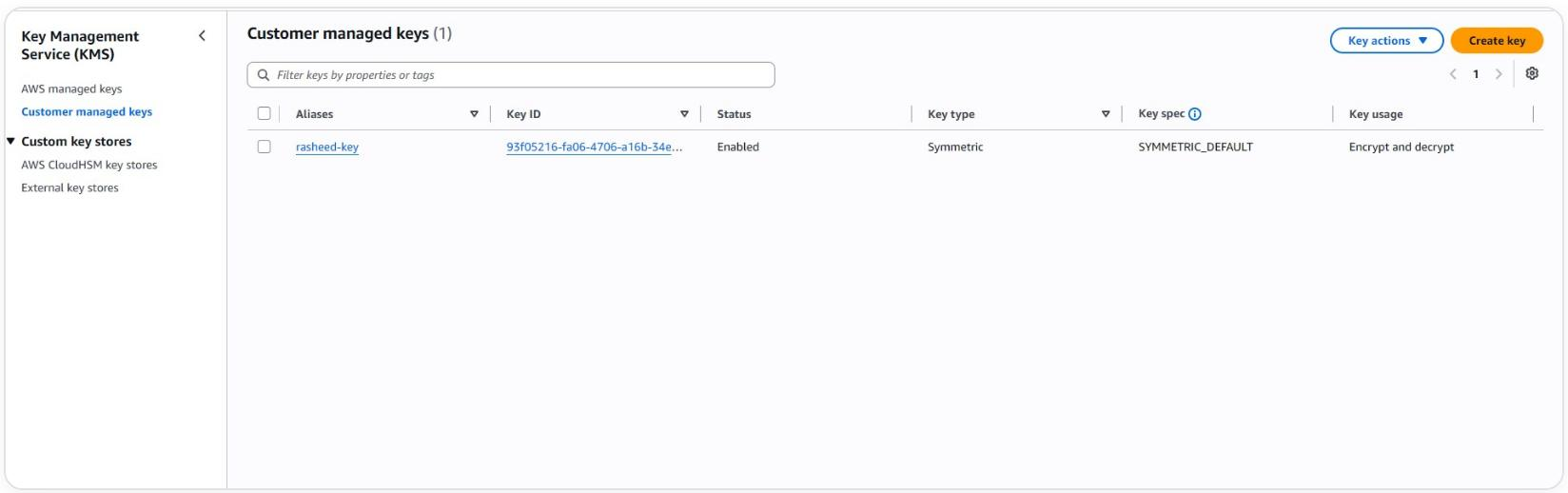
1 Create & Scope the KMS Key

Customer Managed Key · Key Policy

STEP 01

Create a customer managed key

A symmetric customer-managed key, **rasheed-key**, was created in KMS (`93f05216-fa06-4706-a16b-34efda74eff5`), set up for encrypt and decrypt usage. Unlike an AWS-owned key, this key lives in the account and its access is fully controlled by a key policy that we write ourselves.

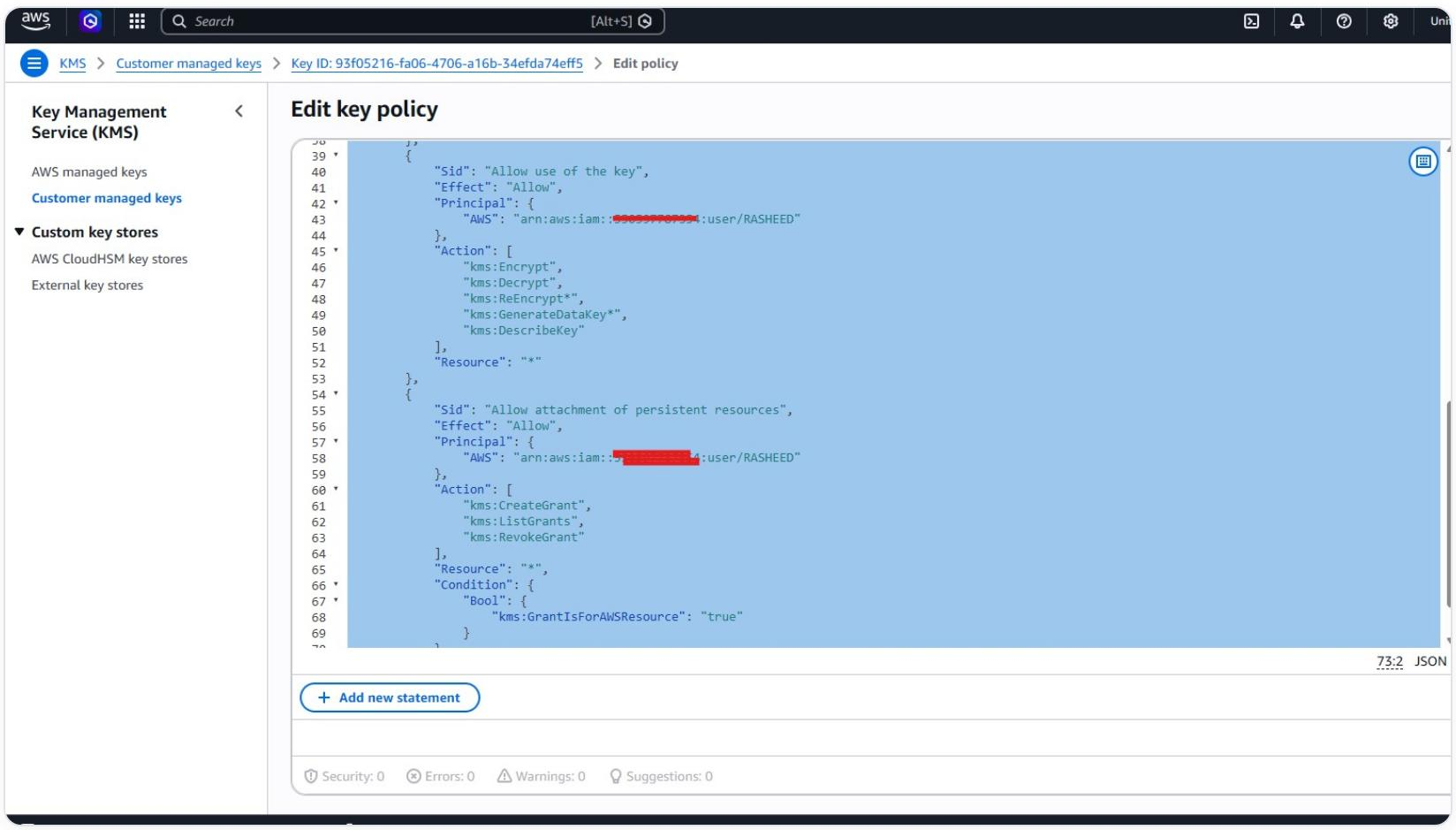


Enabled | Symmetric · SYMMETRIC_DEFAULT

STEP 02

Scope the key policy to one IAM user

The key policy was edited to add a statement naming **arn:aws:iam::<account>:user/RASHEED** as principal, granting `kms:Encrypt` , `kms:Decrypt` , `kms:ReEncrypt*` , `kms:GenerateDataKey*` , and `kms:DescribeKey` , plus grant-management actions for AWS-resource use. No other IAM identity is named — meaning by default, nobody else can use this key at all.



Principal: user/RASHEED | Encrypt · Decrypt · GenerateDataKey

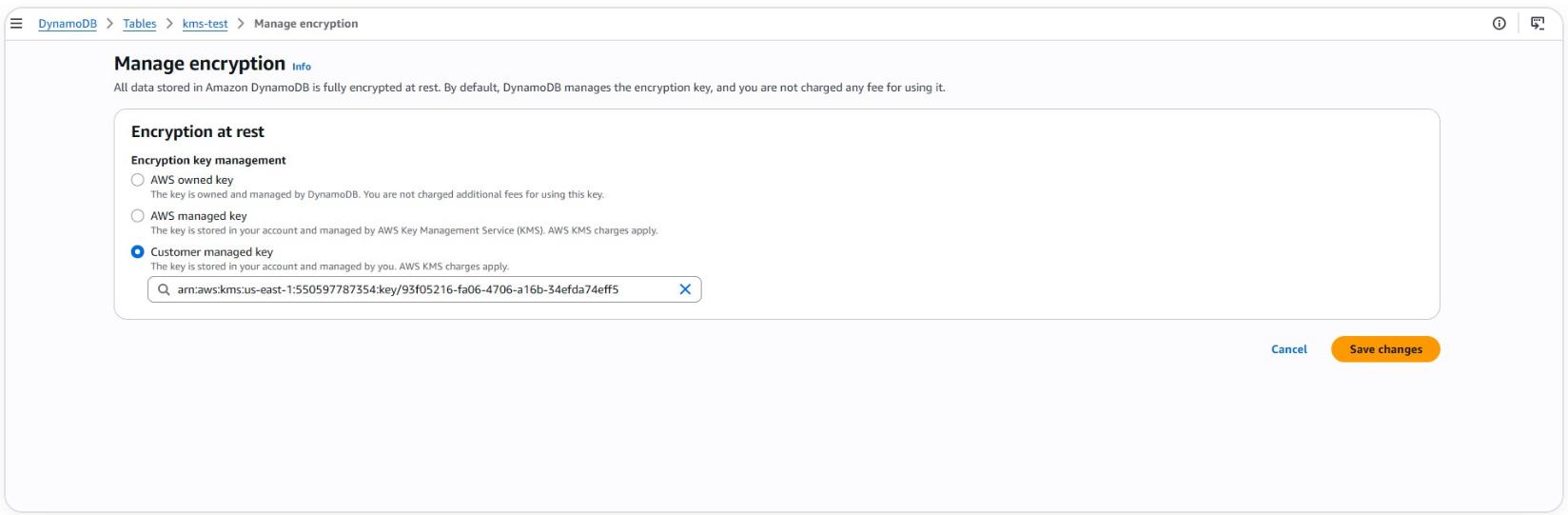
2 Attach the Key & Validate the Boundary

DynamoDB Encryption · Access Denied Test

STEP 03

Encrypt the table with the customer managed key

On the **kms-test** table, under **Manage encryption**, the encryption key management option was switched from the default AWS owned key to **Customer managed key**, and the ARN of `rasheed-key` was selected. From this point on, every read of this table's data requires a successful `kms:Decrypt` call against that key — not just DynamoDB table permissions.

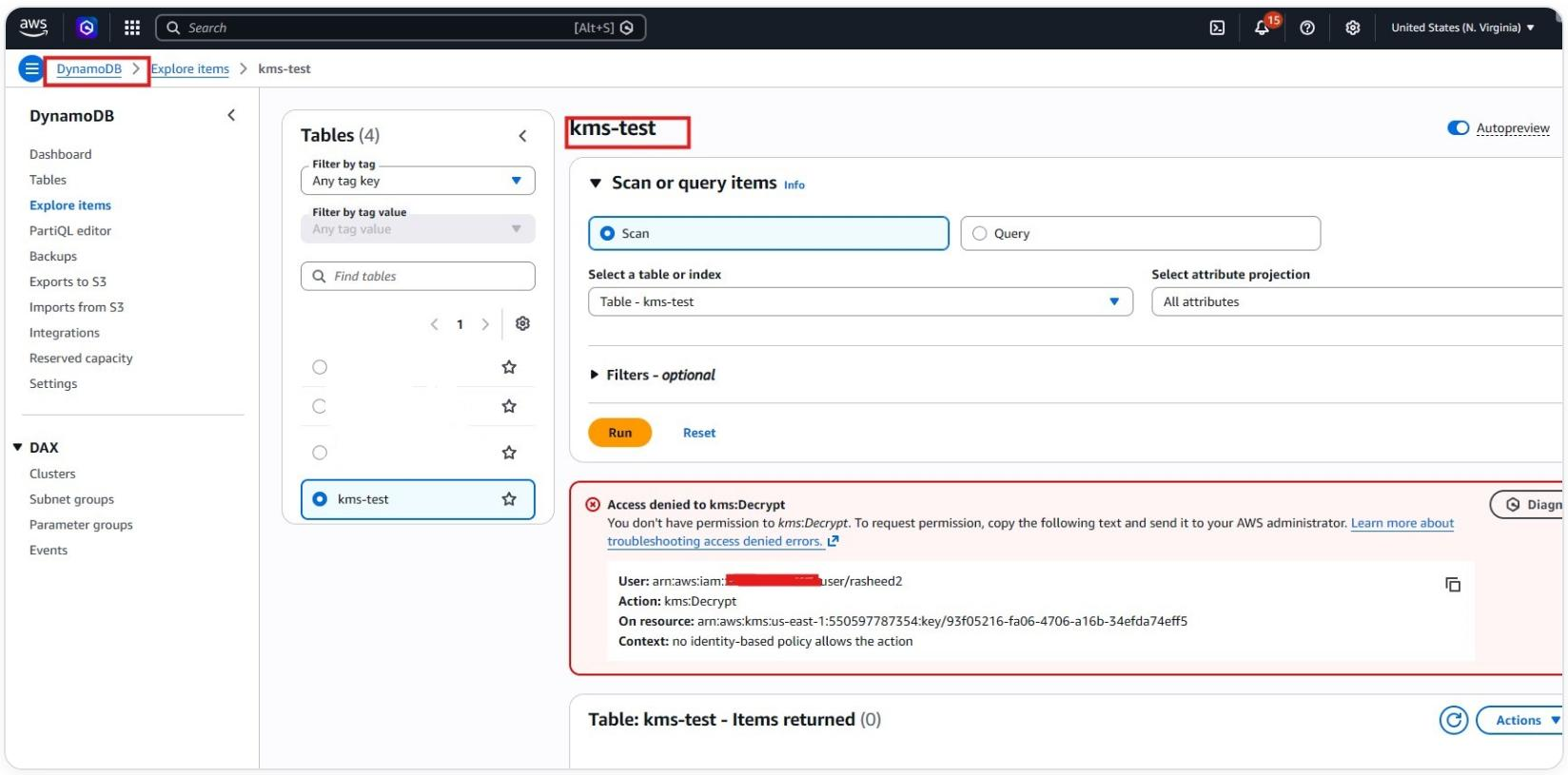


Customer managed key attached

STEP 04

Test as the unauthorized user — access denied

Logged in as **rasheed2** — an IAM user with full DynamoDB permissions — running a **Scan** on **kms-test** fails with **"Access denied to kms:Decrypt."** The error is explicit: `no identity-based policy allows the action` on the key's ARN. DynamoDB permissions were never the problem; the key policy simply never named this user.



AccessDenied — kms:Decrypt

0 items returned

Conclusion & Key Takeaways

Successfully layered a customer-managed KMS key underneath a DynamoDB table so that data access now depends on two independent approvals — the IAM policy for the table, and the key policy for decryption — and proved with a real, denied scan that having one without the other isn't enough.

Key Policy Is a Second Gate

A customer-managed key's policy is evaluated independently of IAM — a user can have full table access and still be denied if the key never named them.

Encryption Becomes Access Control

Attaching the key to the table turns encryption at rest into a genuine authorization boundary, not just a compliance/security checkbox.

Least Privilege by Design

Only the principal explicitly listed in the key policy — RASHEED — can encrypt or decrypt; every other identity is blocked by default.

Validated, Not Assumed

The boundary was proven with a live scan from a second, fully-permissioned IAM user — denied in practice, not just in theory.